

MixBytes()

Lido GGV Migrator Security Audit Report

APRIL 24, 2026

Table of Contents

1. Introduction	2
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	4
1.4 Security Assessment Methodology	6
1.5 Risk Classification	8
1.6 Summary of Findings	9
2. Findings Report	10
2.1 Critical	10
2.2 High	10
2.3 Medium	10
2.4 Low	10
L-1 calculateSteps does not model intra-loop LTV drift, so migrate() can revert with an opaque Aave error mid-run	10
L-2 kill() can be called after a successful migration, emitting a misleading Killed event	12
3. About MixBytes	13

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`GGVMigrator` is a one-shot operator-driven utility that migrates a leveraged Aave V3 position (`weETH` collateral / `variableDebtEthWETH` debt) from Lido's Golden Goose Vault (GGV, a Veda BoringVault at `0xef417FCE1883c6653E7dC6AF7c6F85CCDE84Aa09`) into the Mellow `strETH` subvault at `0x893aa69FBAA1ee81B536f0FbE3A3453e86290080`. The migration is performed atomically within a single `migrate()` transaction that: (optionally) pre-repays part of the subvault's existing WETH debt to free borrowing capacity, iteratively borrows WETH on the subvault, forwards it to GGV to repay GGV's WETH debt and withdraw `weETH` collateral directly to the subvault, and re-supplies that `weETH` as subvault collateral. All hardcoded addresses (Aave V3 Pool, Aave Oracle, aTokens, `WETH`, `weETH`, GGV, `strETH` subvault) are validated against mainnet.

The codebase was audited in 2 days by 3 auditors using a combination of manual review, on-chain state validation, and a fuzz suite executed against a mainnet fork at block `24948365`.

`GGVMigrator` is a single-use contract with multiple layered defences: health-factor checks on both GGV and the subvault before and after the loop (`GGV_MIN_ALLOWED_HF_D18 = 1.01`, `MELLOW_MIN_ALLOWED_HF_D18 = 1.01`); a cumulative-equity-drift bound of \$1 in Aave's 8-decimal base (`MAX_ALLOWED_CUMULATIVE_ERROR = 1e8`); a hard 30-iteration cap; a 1 gwei dust ceiling on GGV's residual WETH debt post-migration; and a reentrancy/re-execution guard via the `migrated` flag. The updated commit (`da932764`) addresses the pre-review blockers from the prior scope with four material changes: (1) `MELLOW_MIN_ALLOWED_HF_D18` lowered from `1.07` to `1.01` because the target subvault's on-chain health factor at the audited block is ~ 1.031 (below the old `1.07` floor), so `migrate()` would have reverted with `HealthFactorIsTooLow` on the pre-loop check; the new `1.01` floor matches `GGV_MIN_ALLOWED_HF_D18` and preserves an above-liquidation safety margin on both sides; (2) `calculateSteps` now bounds `maxBorrowWeth` by available Aave WETH liquidity (`saturatingSub(aToken.totalSupply, debtToken.totalSupply)`), reverting `TooManyIterations(uint256.max)` when liquidity is zero; (3) `migrate()` gains optional `wethToRepay` and `wethToBorrow` legs so the operator can pre-fund repayment and restore subvault working liquidity atomically; (4) the `migrated` flag is now set before any external call, hardening the contract against re-execution and theoretical reentrancy paths.

No Critical, High, or Medium issues were identified. Two Low-severity findings were raised. The most practical constraint surfaced by the audit is a market condition rather than a code defect: at the reviewed block, on-chain validation of the state read by `GGVMigrator` at lines 50-89 (block `24948365`) shows GGV WETH debt of $\sim 105,203$ WETH against $\sim 105,015$ `weETH` of collateral, while the Aave WETH market has `debtToken.totalSupply` exceeding

`aToken.totalSupply` by ~281 WETH (i.e. fully utilized), so `saturatingSub(aToken.totalSupply, debtToken.totalSupply)` yields 0 borrowable WETH. As a result, `calculateSteps(maxUtilizationD18)` reverts with `TooManyIterations(type(uint256).max)` for any `maxUtilizationD18` when `wethToRepay = 0`; the minimum `wethToRepay` needed to bring iterations under the 30-cap depends on `maxUtilizationD18`: at `util ≈ 0.99` the Aave-liquidity term binds and ~3,800 WETH suffice, at `util = 0.40` the subvault-headroom term binds and ~8,300 WETH are needed, and at `util = 0.30` the bound rises to ~11,200 WETH (the happy-path fuzz therefore ranges `wethToRepay` over 15,000–30,000 WETH to stay clear of the boundary). `MAX_ALLOWED_ITERATIONS = 30` is the correct protection, but the current parameter set does not surface the constraint to the operator without reading Aave state off-chain. The operator must either wait for Aave WETH liquidity to free up, or fund the subvault with substantial WETH before calling `migrate()`.

Rounding behavior was reviewed separately and is well-directed. `calculateSteps` has four division sites: `maxBorrowValue * maxUtilizationD18 / 1 ether` and `maxBorrowValue * 1 ether / wethPrice` both round down in favor of stricter per-iteration caps; `iterations = ceilDiv(wethDebt, maxBorrowWeth)` rounds up so the loop can always retire the full debt; `wethPerStep = wethDebt / iterations` rounds down, leaving at most `iterations - 1 ≤ 29` wei of GGV residual debt – well inside the `MAX_WETH_DEBT_AFTER_MIGRATION = 1 gwei` ceiling. The one counterparty-facing site is `weethPerStep = mulDiv(wethPerStep, wethPrice, weethPrice)`, which also rounds down. Fuzzing was performed across the precision-loss path and confirms the drift stays inside the bound.

Out of scope for that audit: Aave V3 protocol, the Mellow flexible-vaults framework (subvault, CallModule, VerifierModule), the Veda BoringVault implementation, and any off-chain components (operator tooling, merkle-root generation, keepers).

1.3 Project Overview

Summary

Title	Description
Client	Lido
Category	Liquid Staking
Project	GGV Migrator
Type	Solidity
Platform	EVM
Timeline	23.04.2026 - 24.04.2026

Scope of Audit

File	Link
src/utils/GGVMigrator.sol	src/utils/GGVMigrator.sol

Versions Log

Date	Commit Hash	Note
23.04.2026	ea7ec360e579816a93d3f3813b762fbf04be062e	Initial Commit
24.04.2026	da932764a3e9fbc71127f39aa1006a2ddf73d46c	Commit with Updates

Mainnet Deployments

File	Address
GGVMigrator.sol	0x000000000433bE90C51EE86E801C65977cFb42d9F

The deployment was reviewed and its configuration matches the audited scope. Deployed bytecode is compiled from the [da932764a3e9fbc71127f39aa1006a2ddf73d46c](#) commit and the hardcoded `SUBVAULT` constant resolves to [0x893aa69FBAA1ee81B536f0FbE3A3453e86290080](#) as expected. The owning Safe uses the canonical Gnosis Safe v1.3.0 singleton ([0xd9Db270c1B5E3Bd161E8c8503c55cEABeE709552](#)), threshold `5`, `8` owners (all EOAs, no nested Safes or controller contracts), nonce `22` (active operational use), zero modules installed (no bypass paths around the 5-of-8 approval), and no Guard contract. Compromise of the owning Safe is unlikely under this configuration (5-of-8 threshold across independent EOA signers, no modules to bypass it, and canonical Safe contracts).

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	Project Architecture Review: • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: • Cross-check findings among auditors • Discuss identified issues • Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	Bug Fixing & Re-Audit: • The client addresses the identified issues and provides feedback • Auditors verify the fixes and update their statuses with supporting evidence • A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	Final Code Verification & Public Audit Report: • Verify the final code version against recommendations and their statuses • Check deployed contracts for correct initialization parameters • Confirm that the deployed code matches the audited version • Issue a public audit report, published on our official GitHub repository • Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	0
Low	2

Findings Statuses

ID	Finding	Severity	Status
L-1	<code>calculateSteps</code> does not model intra-loop LTV drift, so <code>migrate()</code> can revert with an opaque Aave error mid-run	Low	Acknowledged
L-2	<code>kill()</code> can be called after a successful migration, emitting a misleading <code>Killed</code> event	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

Not Found

2.4 Low

L-1	calculateSteps does not model intra-loop LTV drift, so migrate() can revert with an opaque Aave error mid-run		
Severity	Low	Status	Acknowledged

Description

In `src/utils/GGVMigrator.sol` at lines 50-89, `calculateSteps` sizes each iteration from a single snapshot of the subvault's borrowing headroom taken at the entry of `migrate()`: `maxBorrowWeth = maxBorrowValue * maxUtilizationD18 / wethPrice`, then divides the GGV debt into equal-sized steps. The loop at lines 130-146 executes each step as `AAVE.borrow(WETH, wethPerStep)` on the subvault, followed by a WETH transfer to GGV, a GGV-side repay, a `withdraw(WEETH, ...)` back to the subvault, and only then a `supply(WEETH, ...)` that restores the subvault's collateral. Between the borrow and the subsequent supply the subvault's debt has grown but its collateral has not, so its effective LTV ratchets upward on every iteration. The pre-flight check uses the initial, lower LTV and does not simulate this drift.

When the operator picks a `maxUtilizationD18` that is valid at entry but leaves insufficient margin for the cumulative drift, a later iteration's `borrow` fails Aave's post-borrow LTV check against the eMode Category 1 cap (LTV 9300 bps) and reverts with `CollateralCannotCoverNewBorrow`. Fuzzing against a mainnet fork confirms this happens for many `maxUtilizationD18` values inside the accepted `(0, 1e18)` range. The migrator does not wrap the Aave call, so the failure bubbles up as an unnamed low-level revert with no indication of which step failed or how much headroom was missing.

A failed transaction reverts all state changes (including `migrated = true`), so the operator can retry with a smaller `maxUtilizationD18`; the cost is the gas of the failed run and operator time spent guessing a safe value. No user-funds risk.

Recommendation

1. Wrap the per-iteration Aave `borrow` call in `try/catch` and re-revert with a typed `AaveBorrowRejected(uint256 stepIndex, bytes4 innerSelector)` so the operator sees which

step failed and why.

2. Document the safe operational range for `maxUtilizationD18` in the runbook. The accepted `(0, 1e18)` range is input validation, not guidance; fuzzing against the block-`24948365` fork shows `[0.30 ether, 0.40 ether]` completes reliably for the current subvault state.

L-2	<code>kill()</code> can be called after a successful migration, emitting a misleading <code>Killed</code> event		
Severity	Low	Status	Acknowledged

Description

In `src/utils/GGVMigrator.sol` at lines 91-94, `kill()` unconditionally sets `migrated = true` and emits `Killed(block.timestamp, caller)`. There is no check that `migrated` was previously `false`. After a successful `migrate()` run, an `owner` call to `kill()` succeeds and emits a `Killed` event, which can confuse off-chain indexers that treat `Killed` and `Migrated` as mutually exclusive states. The impact is limited to event-log hygiene, with no functional effect on migration correctness or fund safety.

Recommendation

Add a `require(!migrated)` guard in `kill()` and revert with `AlreadyMigrated()` (or a dedicated `AlreadyKilled()` error) when called after the flag has already been set.

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**, **Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information



<https://mixbytes.io/>



https://github.com/mixbytes/audits_public



<https://x.com/MixBytes>



hello@mixbytes.io